EXP NO: 6	*FEEDFORWARD AND CONVOLUTIONAL NEURAL NETWORKS*

AIM:

To build, train and compare a Feedforward Neural Network (Multi-Layer Perceptron) and a Convolutional Neural Network (CNN) for classifying handwritten digit images.

ALGORITHM:

1. Import the required libraries (numpy, matplotlib, seaborn, scikit-learn).
2. Load the Digits dataset (8x8 images, 10 classes), scale the pixel values to 0-1 and split into training and testing sets.
3. Feedforward network: build an MLP with hidden layers of 64 and 32 neurons (ReLU, Adam optimiser) using MLPClassifier and train it.
4. CNN: build the network Conv(8 filters, 3x3) -> ReLU -> MaxPool(2x2) -> Flatten -> Dense(32) -> Softmax(10) from scratch using NumPy.
5. Train the CNN with forward propagation, cross-entropy loss, backpropagation and the Adam optimiser for 30 epochs.
6. Evaluate both networks on the test data using accuracy, classification report and confusion matrix.
7. Compare the accuracy and the number of parameters of the two networks.
8. Plot the training loss curves, the CNN confusion matrix, the accuracy comparison and the learned convolution filters.

CODE:

```
# 1. Import libraries
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from numpy.lib.stride_tricks import sliding_window_view
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,

    classification_report

np.random.seed(42)
# 2. Load dataset (8x8 handwritten digit images, 10 classes) and split
```

```
digits = load_digits()
X, y = digits.data / 16.0, digits.target          # pixel values scaled to 0-1
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print("Training samples:", X_train.shape[0], " Test samples:", X_test.shape[0])

# ====================== PART A: FEEDFORWARD NEURAL NETWORK ======================
# 3. Build and train a feedforward network (64 -> 64 -> 32 -> 10)
ffnn = MLPClassifier(hidden_layer_sizes=(64, 32), activation='relu',
    solver='adam',
                     learning_rate_init=0.001, max_iter=200, random_state=42)
ffnn.fit(X_train, y_train)
ffnn_params = sum(w.size for w in ffnn.coefs_) + sum(b.size for b in
    ffnn.intercepts_)
ffnn_acc = accuracy_score(y_test, ffnn.predict(X_test))
print("\n[Feedforward NN] Architecture: 64-64-32-10    Parameters:",
    ffnn_params)
print("[Feedforward NN] Test Accuracy:", round(ffnn_acc, 4))

# ====================== PART B: CONVOLUTIONAL NEURAL NETWORK ======================
# CNN built from scratch with NumPy:
# Input(8x8x1) -> Conv(8 filters, 3x3, ReLU) -> MaxPool(2x2) -> Flatten(128)
#              -> Dense(32, ReLU) -> Dense(10, Softmax)
F = 8
def he(shape, fan_in):
    return np.random.randn(*shape) * np.sqrt(2.0 / fan_in)
P = {'Wc': he((9, F), 9), 'bc': np.zeros(F),
     'W1': he((4 * 4 * F, 32), 4 * 4 * F), 'b1': np.zeros(32),
     'W2': he((32, 10), 32), 'b2': np.zeros(10)}
cnn_params = sum(v.size for v in P.values())
print("\n[CNN] Architecture: Conv(8@3x3) -> MaxPool(2x2) -> Dense(32) ->
    Dense(10)   Parameters:", cnn_params)

relu = lambda z: np.maximum(0, z)

def forward(Xb):
    N = Xb.shape[0]
    img = Xb.reshape(N, 8, 8, 1)
    pad = np.pad(img, ((0, 0), (1, 1), (1, 1), (0, 0)))
    cols = sliding_window_view(pad, (3, 3), axis=(1, 2)).reshape(N * 64, 9)   #
        im2col
    z1 = cols @ P['Wc'] + P['bc']
    a1 = relu(z1).reshape(N, 8, 8, F)
    r = a1.reshape(N, 4, 2, 4, 2, F)
    pooled = r.max(axis=(2, 4))                                               #
        max pooling
    flat = pooled.reshape(N, -1)
    z2 = flat @ P['W1'] + P['b1']; a2 = relu(z2)
    z3 = a2 @ P['W2'] + P['b2']
    e = np.exp(z3 - z3.max(axis=1, keepdims=True)); probs = e / e.sum(axis=1,
        keepdims=True)
    return probs, (cols, z1, r, pooled, flat, z2, a2)

def backward(probs, cache, Yb):
    cols, z1, r, pooled, flat, z2, a2 = cache
```

```
    N = Yb.shape[0]
    g = {}
    dz3 =(probs - Yb) /N
    g['W2'] =a2.T  @ dz3;g['b2'] = dz3.sum(0)
    dz2 =(dz3  @ P['W2'].T)*(z2 > 0)
    g['W1'] =flat.T  @ dz2;g['b1'] = dz2.sum(0)
    dpool=  (dz2 @ P['W1'].T).reshape(N, 4, 4, F)
    mask =  (r== pooled[:,:,None, :, None, :]).astype(float)
    mask /= mask.sum(axis=(2,4), keepdims=True)
    da1 =(mask  *dpool[:,:,None, :, None, :]).reshape(N * 64, F)
    dz1 =da1* (z1 >0)
    g['Wc'] =cols.T  @ dz1;g['bc'] = dz1.sum(0)
    return g

# Adam optimiser
m = {k: np.zeros_like(v) for k, v in P.items()}; v_ = {k: np.zeros_like(v) for
    k, v in P.items()}
t, lr = 0, 0.005
def adam_step(g, b1=0.9, b2=0.999, eps=1e-8):
    global t
    t += 1
    for k in P:
        m[k] = b1 * m[k] + (1 - b1) * g[k]; v_[k] = b2 * v_[k] + (1 - b2) *
            g[k] ** 2
        P[k] -= lr * (m[k] / (1 - b1 ** t)) / (np.sqrt(v_[k] / (1 - b2 ** t)) +
            eps)

# 4. Train the CNN
Y_train = np.eye(10)[y_train]
EPOCHS, BATCH = 30, 32
loss_hist, acc_hist = [], []
for epoch in range(1, EPOCHS + 1):
    idx = np.random.permutation(len(X_train)); total = 0
    for i in range(0, len(idx), BATCH):
        b = idx[i:i + BATCH]
        probs, cache = forward(X_train[b])
        total += -np.sum(Y_train[b] * np.log(probs + 1e-9))
        adam_step(backward(probs, cache, Y_train[b]))
    loss_hist.append(total / len(X_train))
    acc_hist.append(accuracy_score(y_test, forward(X_test)[0].argmax(1)))
    if epoch % 5 == 0 or epoch == 1:
        print(f"[CNN] Epoch {epoch:2d}/{EPOCHS}  Loss: {loss_hist[-1]:.4f}
            Test Accuracy: {acc_hist[-1]:.4f}")

# 5. Evaluate the CNN
y_pred_cnn = forward(X_test)[0].argmax(1)
cnn_acc = accuracy_score(y_test, y_pred_cnn)
print("\n[CNN] Test Accuracy:", round(cnn_acc, 4))
print("\nCNN Classification Report:\n", classification_report(y_test,
    y_pred_cnn))

# 6. Compare both networks
print("Comparison:")
print(f" Feedforward NN : Accuracy = {ffnn_acc:.4f}    Parameters =
    {ffnn_params}")
```

```
print(f" CNN             :Accuracy = {cnn_acc:.4f}    Parameters =
    {cnn_params}")

# 7. Visualisation: loss curves, confusion matrix and learned filters
fig, ax = plt.subplots(1, 3, figsize=(15, 4))
ax[0].plot(ffnn.loss_curve_, label='Feedforward NN'); ax[0].plot(loss_hist,
    label='CNN')
ax[0].set_xlabel("Epoch"); ax[0].set_ylabel("Training Loss");
    ax[0].set_title("Training Loss Curves"); ax[0].legend()
sns.heatmap(confusion_matrix(y_test, y_pred_cnn), annot=True, fmt='d',
    cmap='Blues', ax=ax[1], cbar=False)
ax[1].set_title("CNN Confusion Matrix"); ax[1].set_xlabel("Predicted");
    ax[1].set_ylabel("Actual")
ax[2].bar(['Feedforward NN', 'CNN'], [ffnn_acc, cnn_acc], color=['steelblue',
    'darkorange'])
ax[2].set_ylim(0.9, 1.0); ax[2].set_ylabel("Test Accuracy");
    ax[2].set_title("Accuracy Comparison")
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 2.8))
for i in range(F):
    plt.subplot(1, F, i + 1)
    plt.imshow(P['Wc'][:, i].reshape(3, 3), cmap='gray'); plt.axis('off');
        plt.title(f"F{i + 1}")
plt.suptitle("Learned 3x3 convolution filters")
plt.show()
```

OUTPUT:

```
Training samples: 1437                    Test samples: 360

[Feedforward NN]  Architecture: 64-64-32-10   Parameters: 6570
[Feedforward NN]  Test Accuracy: 0.9778

[CNN] Architecture: Conv(8@3x3) -> MaxPool(2x2) -> Dense(32) -> Dense(10)
    Parameters: 4538
[CNN] Epoch  1/30  Loss: 1.4009  Test Accuracy:  0.7750
[CNN] Epoch  5/30  Loss: 0.1162  Test Accuracy:  0.9611
[CNN]  Epoch  10/30 Loss: 0.0481  Test Accuracy:  0.9722
[CNN]  Epoch  15/30 Loss: 0.0211  Test Accuracy:  0.9722
[CNN]  Epoch  20/30 Loss: 0.0134  Test Accuracy:  0.9806
[CNN]  Epoch  25/30 Loss: 0.0052  Test Accuracy:  0.9833
[CNN] Epoch 30/30  Loss: 0.0027 Test Accuracy: 0.9806

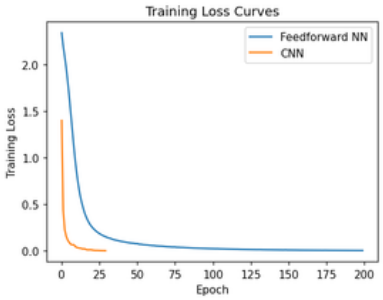

[CNN] Test Accuracy: 0.9806

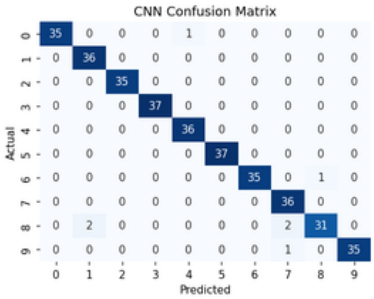
CNN Classification Report:
              precision    recall  f1-score   support

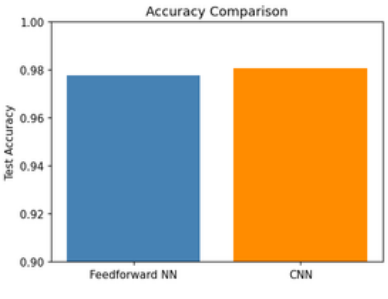
           0       1.00      0.97      0.99        36
           1       0.95      1.00      0.97        36
           2       1.00      1.00      1.00        35
           3       1.00      1.00      1.00        37
           4       0.97      1.00      0.99        36
           5       1.00      1.00      1.00        37
           6       1.00      0.97      0.99        36
           7       0.92      1.00      0.96        36
           8       0.97      0.89      0.93        35
           9       1.00      0.97      0.99        36

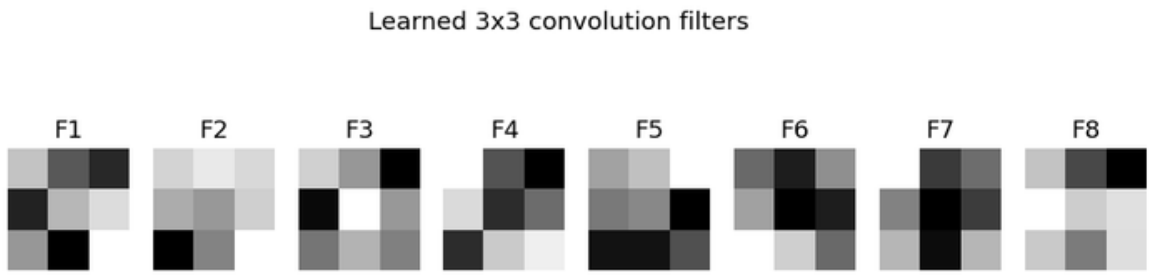

    accuracy                           0.98       360
macro    avg       0.98      0.98      0.98       360
weighted avg       0.98      0.98      0.98       360

Comparison:
  Feedforward NN : Accuracy = 0.9778   Parameters = 6570
  CNN            : Accuracy = 0.9806   Parameters = 4538
```









RESULT:

A Feedforward Neural Network and a Convolutional Neural Network were successfully built and trained on the handwritten digits dataset. The feedforward network (6,570 parameters) achieved a test accuracy of 97.78%, while the CNN (4,538 parameters) achieved 98.06%. The CNN reached a slightly higher accuracy with fewer parameters because its shared convolution filters capture local patterns in the images.